Engineering Journal

WRO 2026 Future Engineers — Self-Driving Cars

Team The Red Castle | HMK AI and Robotics Club

Coach: Ahmad Kalthom | Jolian Wassof · Omar Shammout · Louay Rashwan

Table of Contents

1. Introduction

Self-driving cars are one of the clearest places where robotics, computer vision and control theory meet in a single physical object that either works or doesn't. For WRO 2026 Future Engineers, our goal was to build a vehicle that finds its own way around a track it has never measured, using only what a camera can see.

We built our car around a Raspberry Pi 4 and a single wide-angle camera. No LiDAR, no ultrasonic rangefinders, no wheel encoders, no IMU. Everything the rules require the car to know — where the walls are, which way to lap, where the traffic signs are, where to park — is visible to a camera, so we put our engineering time into making that one sensor trustworthy instead of into fusing several imperfect ones.

That decision shaped most of what follows. A camera-only car lives or dies on image quality and threshold tuning, and a large part of this document is the story of getting those right: a sensor swap that fixed a depth-of-field problem we didn't expect, a wall detector that turned out to be reading corner lines as walls, and a control law that we rebuilt once we had a correct measurement to calibrate it against.

The vehicle drives the Open Challenge — three laps around a track whose inner wall positions are randomised each round — and the Obstacle Challenge, which adds red and green traffic signs that must be passed on the correct side and a parking manoeuvre at the end.

2. Solution

2.1 Open Challenge

The Open Challenge is three laps of a track whose inner wall positions are randomised each round, so nothing about the corridor width can be hard-coded in advance — the car has to read it from the camera as it drives.

- Direction is decided once, from the first corner line the camera sees, and locked for the rest of the run — whichever colour reads bigger in that frame sets clockwise or counter-clockwise, and the reading is cross-checked against which side of the image has more open wall as a second opinion.

- Between corners, a proportional-derivative controller holds a fixed distance to the outer wall only. We deliberately do not centre between both walls: the inner wall moves every round, so the outer wall is the one reference that stays put.

- At a corner, crossing the driving-direction's line fires a scripted turn — full steering lock held for a bounded time, ending as soon as the way ahead reads clear rather than after a fixed duration.

- A wall emergency always wins, in any state: if either wall gets too close the car escapes proportionally to how close it is, and if both walls are close at once (jammed facing a corner) it commits to a single escape direction instead of wavering between two.

- The car stops once 11 quadrants are counted and the lap timer has run out, so it finishes driving through the last straight instead of halting on the line itself.

2. Solution

2.2 Obstacle Challenge

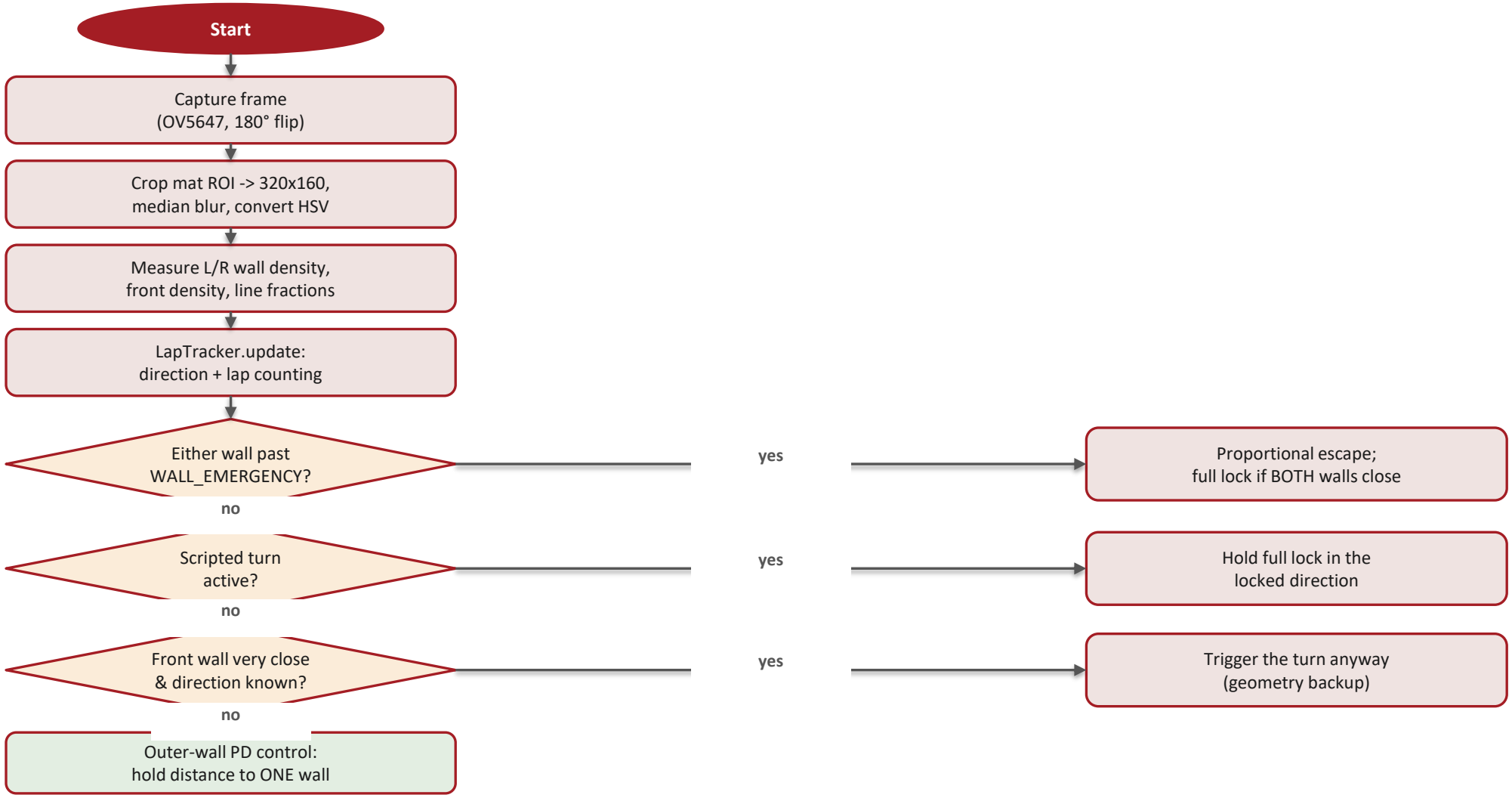
The Obstacle Challenge adds red and green traffic signs that must be passed on the correct side, then a parking manoeuvre once the laps are done.

- If a sign is visible, the car steers to place it correctly — red on the car's right, green on its left — with the target position sliding further toward the frame edge as the sign gets closer, so the manoeuvre curves rather than swerving late.
- If a sign was visible a moment ago but drops out of view for a frame, the last steering command is held rather than reacted to — sign detection can flicker sharply between consecutive frames, and reacting to every drop-out broke passes mid-way.
- Between signs, the car falls back to the same outer-wall lane-keeping law used in the Open Challenge, instead of driving straight and hoping a wall override corrects it in time.
- A wall override sits above all of this and also does the job of turning the car through each corner, since there is no separate scripted-turn sequencer here.
- Parking is not yet wired into the main loop — the magenta gate is currently treated purely as a wall to avoid. This is called out honestly in the Evolution section.

2. Solution

2.3 Control-Loop Flowchart — Sense & Decide

Every frame runs the same read -> decide -> act loop. navigate() in robot.py picks the steering command in this strict priority order.

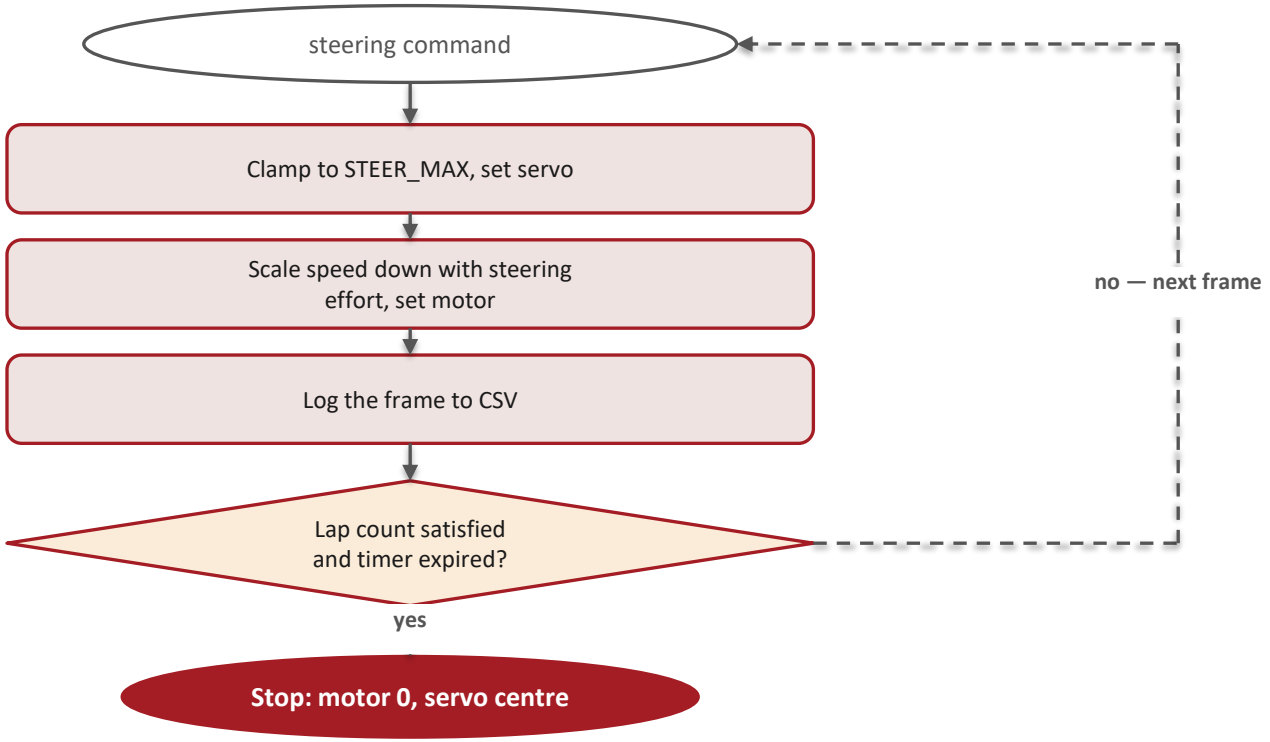


G, I, K or L → the steering command continues on "Control-Loop Flowchart — Act & Loop"

2. Solution

2.3 Control-Loop Flowchart — Act & Loop

Continues from the steering command chosen on the previous slide (G, I, K or L).

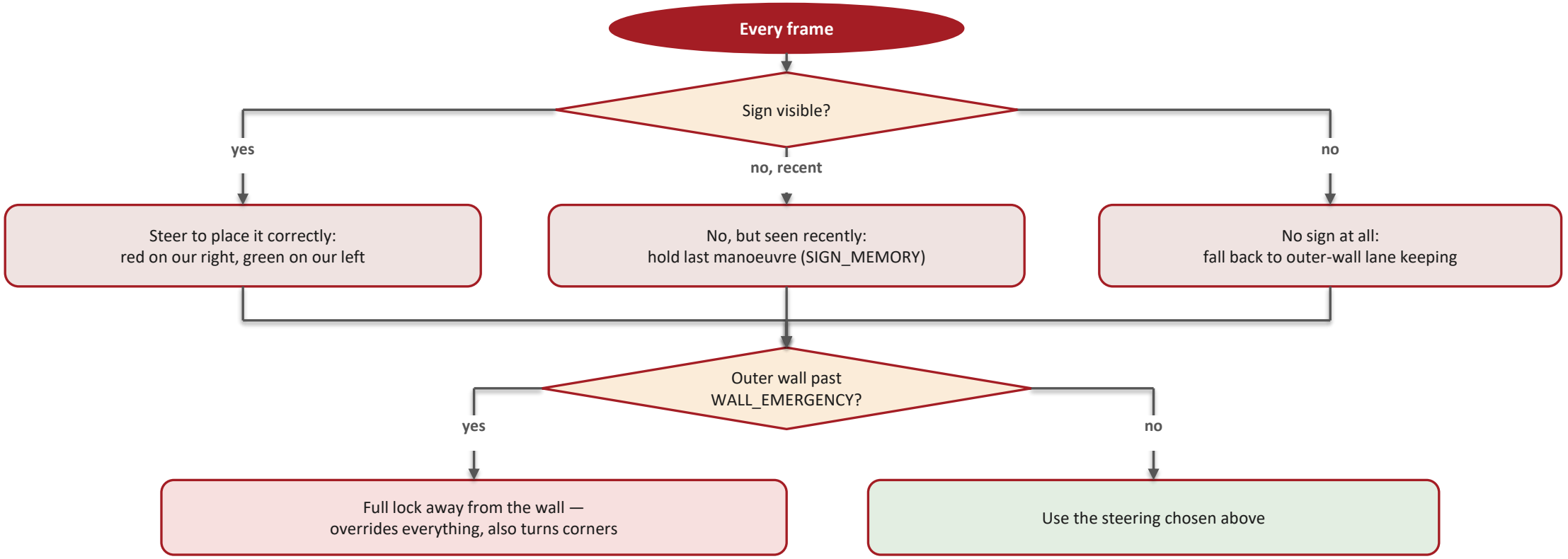


This priority order is what makes a wall emergency unconditional: it is checked first, every frame, before the turn state or the lane-keeping law ever gets a say. The scripted turn and the geometry backup only run when no wall is already too close.

2. Solution

2.4 Obstacle Challenge — Priority Flowchart

Steering is decided by a strict priority, so a lower-priority behaviour can never override safety.



Priority, highest to lowest: (1) wall override — also turns every corner, (2) sign steering, (3) sign-hold memory for detection flicker, (4) outer-wall lane keeping between signs.

3. Mobility Management

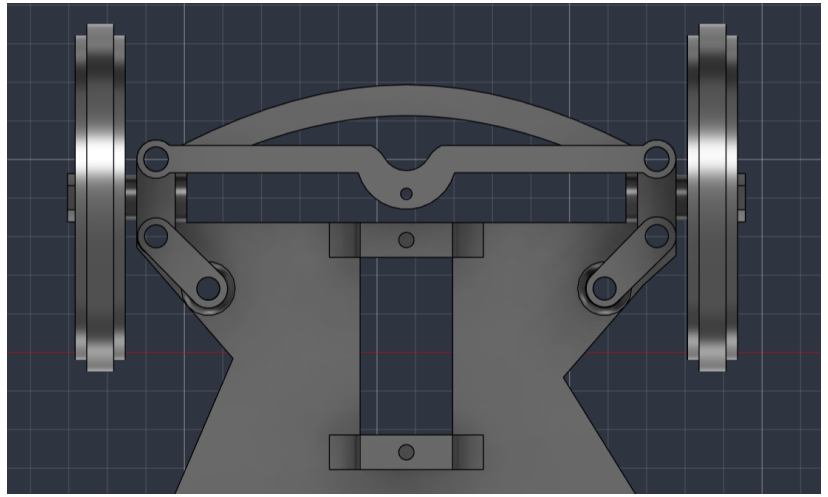
3.1 The Steering Assembly

Steering is Ackermann-style, driven by a single MG90S micro servo (GPIO13, which is a hardware-PWM pin — a smoother, lower-jitter signal than a software-timed one).

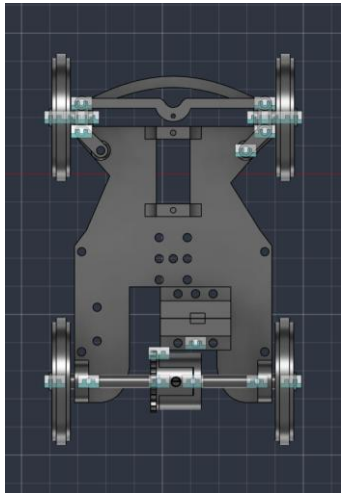
The servo turns a central bell-crank, which pushes a tie-rod out to both front steering knuckles. Because the linkage geometry is Ackermann and not a simple parallel arm, the inner wheel turns through a sharper angle than the outer one in a corner — each wheel follows its own turning radius instead of fighting the other and scrubbing.

The linkage's mechanical limit is ±35°. In software we run it at ±20° (STEER_MAX in config.py) for stability at full driving speed — the wider mechanical range is there if a future tuning pass wants it.

Every steering command in the codebase passes through this one constant and one servo() function, so there is exactly one place that ever needs to know the true centre position — the field-calibrated trim is stored in servo_center.txt (currently -9°) and applied automatically.



Front steering linkage, top view



Steering turned, showing the linkage sweep

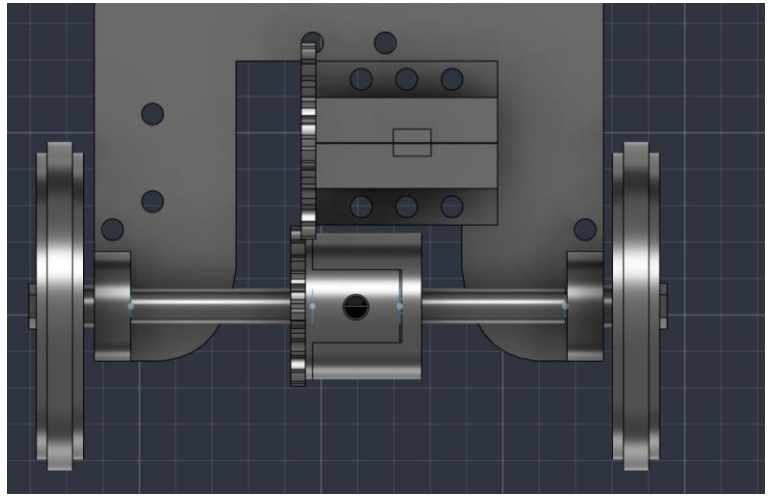
3. Mobility Management

3.2 The Driving Assembly

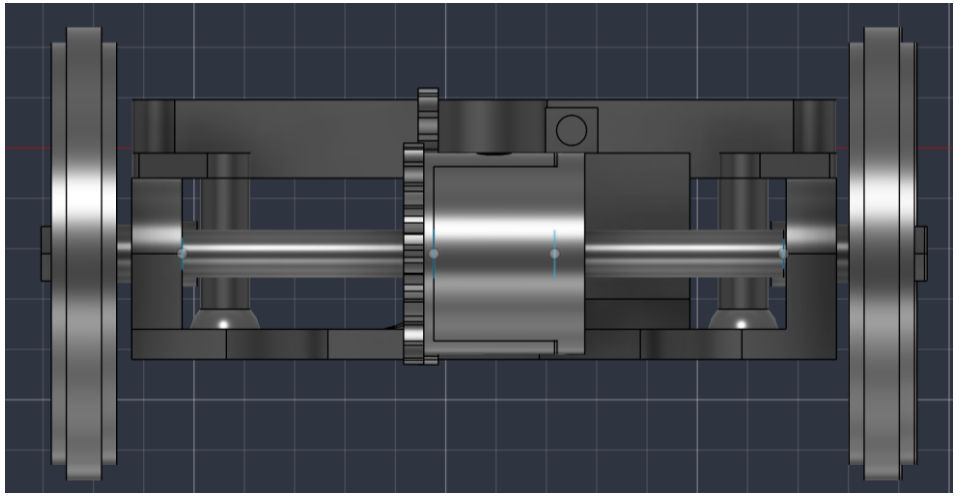
Drive comes from an N20 geared DC motor rated 12 V, 200 rpm, chosen for its compact size and enough torque to pull the car away from rest repeatedly without stalling.

The motor is controlled by an L9110S dual H-bridge, wired so one input is held low while the other carries the PWM speed signal (GPIO24 = forward, GPIO23 = reverse). In software, motor() never switches directly from forward to reverse — it coasts to a stop and waits first, because slamming the direction while the motor is still spinning drives its back-EMF onto the supply rail and can destroy the Pi's regulator. We found this out by losing a regulator early in the build.

From the motor, a 25-tooth pinion drives a 20-tooth gear (a 1.25:1 reduction) into a 3D-printed differential we designed ourselves — outer shell, outer ring gear, and two inner bevel gears on long and short shafts. The differential is the single change that let us raise the steering limit: with a solid axle, the outer wheel scrubbing against the inner one through a corner cost so much traction we had to cap steering near 8° just to keep the car from stalling mid-turn.



N20 motor driving the differential gear mesh

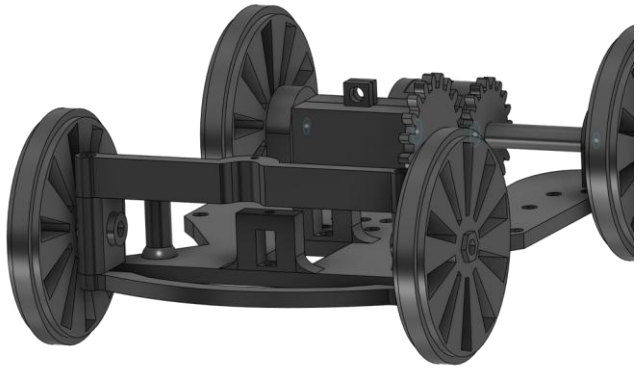


Rear axle, front view

3. Mobility Management

3.3 The Robot's Body / 3D Design

The chassis was designed in Fusion 360 and printed in PLA+ Silk Silver on a Bambu Lab A1: a base plate, a middle plate and a top plate carrying the Pi, battery, driver, buck converter and the camera mast, sized to keep mass low and central over the wheelbase.

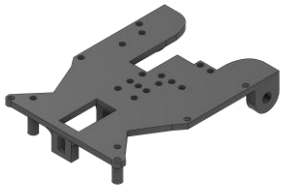


Full car assembly

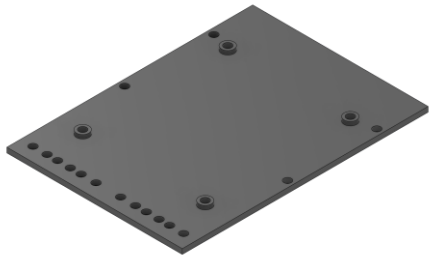
Mass 407 g, overall footprint 14.2 x 9.3 x 15 cm, wheelbase 9.4 cm, rear track 8.5 cm — a wheelbase-to-track ratio close to 1:1, a stable proportion for cornering without tipping risk at this size.

3. Mobility Management

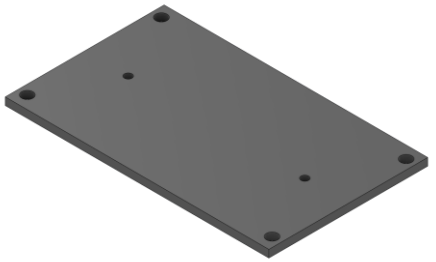
3.3b Printed Parts — Individual Renders (1/2)



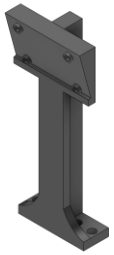
Base



Middle plate

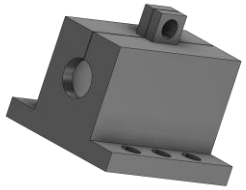


Top plate

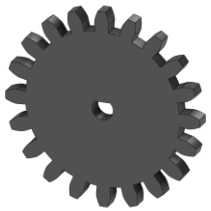


Camera holder

Battery slider



N20 motor holder



N20 gear

Inner differential gear

Outer differential gear

3. Mobility Management

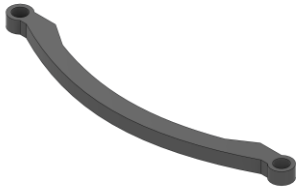
3.3b Printed Parts — Individual Renders (2/2)

Differential outer shell

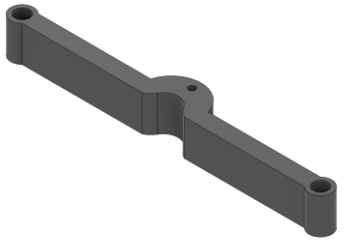
Inner gear, long shaft

Inner gear, short shaft

Ring



Steering base



Steering base servo mount

Steering nut

Steering wheel mount

Wheel

To reason about how fast the car can safely go, we worked through the drivetrain's speed chain from the motor to the ground:

Wheel radius (rw):

$r_w = 4.7\ \text{cm} / 2 = 2.35\ \text{cm} = 0.0235\ \text{m}$

Wheel perimeter (Pw):

$P_w = 2\pi \cdot r_w = 2 \times 3.1416 \times 0.0235 \approx 0.1477\ \text{m}$

Motor speed and the gear reduction:

$\omega_{motor} = 200\ \text{rpm}$ (rated, no load)

$\omega_{axle} = \omega_{motor} / 1.25$ (25:20 reduction) $= 160\ \text{rpm}$

Ground speed (theoretical, no load):

$v = P_w \cdot \omega_{axle} = 0.1477 \times 160 \approx 23.6\ \text{m} \cdot \text{min}^{-1}$

$v \approx 0.394\ \text{m} \cdot \text{s}^{-1}$ (≈ 39 cm/s)

This is a ceiling, not a running speed: tyre friction, the differential, and the car's own 407 g mass all take a share of it. What actually governs how fast the car drives in competition is the 100% cruise setting in software, which cruise_speed() then eases back automatically whenever the steering is working hard — fast on the straights, deliberately slower through a turn.

4. Power and Sense Management

4.1 The Robot's Power Source

Power comes from three 18650 Li-ion cells in series — about 11.1 V nominal, up to 12.6 V fully charged — feeding two separate rails rather than one shared one:

The motor is wired straight from the battery to the L9110S driver's supply pin, with nothing in between. A separate DC-DC step-down converter, rated at 5 V and at least 3 A, powers the Raspberry Pi and the servo on its own rail.

This split exists because of a real failure: early on, the motor and the Pi shared one supply, and the Pi rebooted every time the motor drew current under load — a classic brownout, made worse by the driver's ground not being tied back to the Pi's ground at all, so the only electrical link between them was the GPIO signal wire itself. Separating the rails and tying every ground together (battery, driver, Pi) fixed it completely, and is now a fixed rule on the car rather than a tuning choice.

A rocker switch sits in the battery line as the master on/off. We also note, as a live watch item rather than a solved problem: a freshly charged pack at 12.6 V sits marginally above the L9110S's 12 V rating, so driver temperature is worth checking after a run.

4. Power and Sense Management

4.2 Main Components

- OV5647 wide-angle camera — a small-sensor, fixed-focus module with roughly a 120° field of view. It is the vehicle's only sensor: every wall, corner line, traffic sign and the parking gate is read from its image. Fixed focus means a large depth of field, so the near mat and a far wall stay sharp at the same time with nothing to hunt or mis-lock mid-run — a deliberate choice explained in the Evolution section.

- Raspberry Pi 4 Model B, 2 GB RAM, running Raspberry Pi OS Bookworm — the single board that captures every frame, runs the vision pipeline, decides the steering and speed, and drives the servo and motor through its GPIO header.

- L9110S dual H-bridge motor driver — a compact two-pin-per-motor interface, chosen for its small size and simple PWM control, sitting on its own supply rail straight from the battery.

- MG90S micro servo — ~13 g, metal gear train, powered from the Pi's 5 V rail, driving the Ackermann steering linkage.

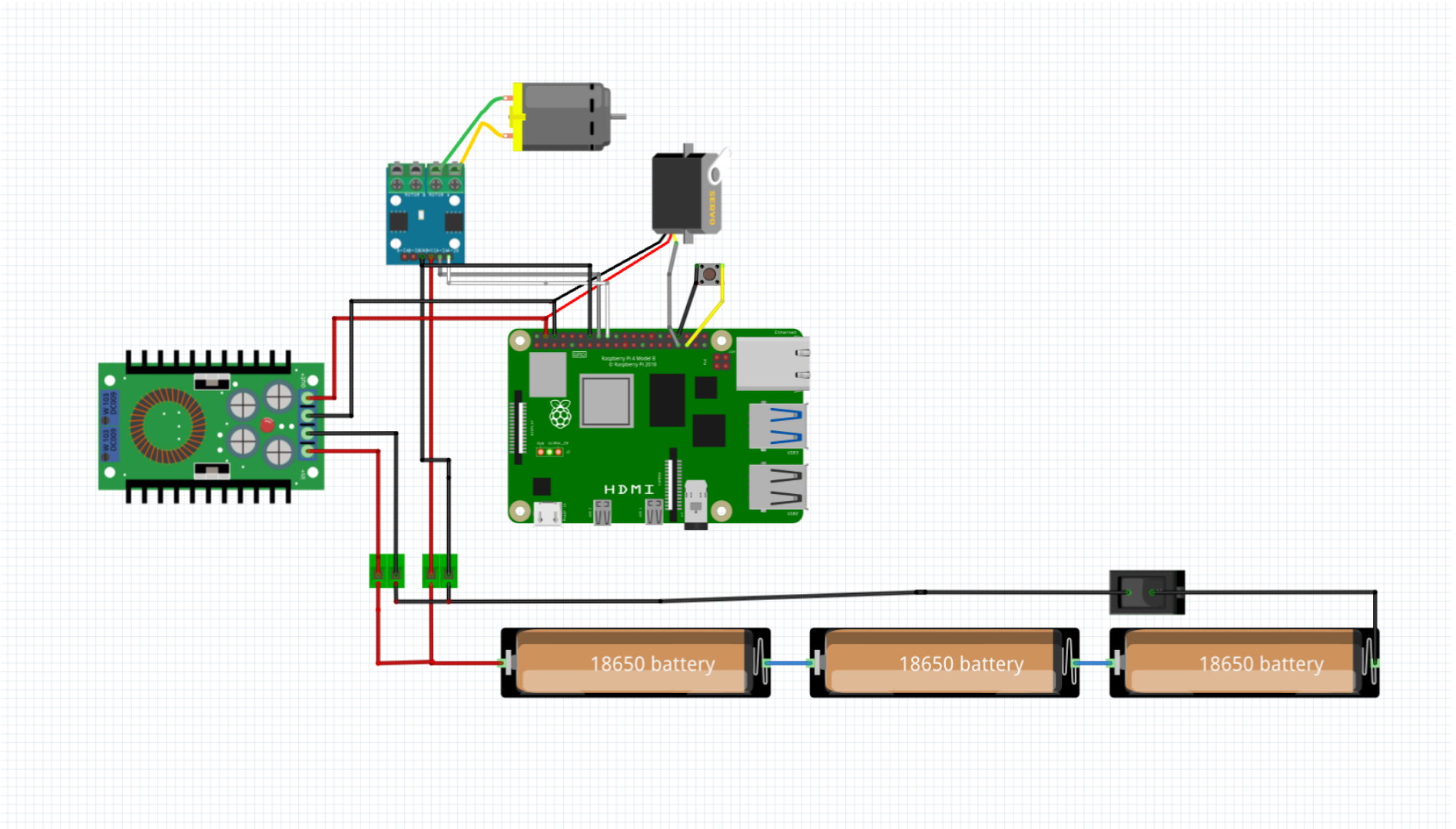
4. Power and Sense Management

4.2b Why We Chose Each Component

Component	What we chose	Alternative considered	Why
Camera	OV5647 fixed-focus, wide FOV	Pi Camera Module 3 (autofocus)	Autofocus hunted and lost distant signs to blur (proof: 5.1a). A ground robot never needs to refocus, so fixed focus cost us nothing and bought a depth of field wide enough for the near mat and a far sign at once.
Motor	N20, 12 V, 200 rpm geared DC	Same footprint, higher-rpm N20	A faster variant trades torque for speed; we needed the torque to pull the 407 g car from rest repeatedly without stalling more than we needed extra top speed.
Driver	L9110S dual H-bridge	L298N dual H-bridge	L298N's extra current headroom is wasted on a single N20 and it takes far more board space; L9110S needs only two GPIO pins and fits the chassis footprint we had.
Servo	MG90S micro servo, GPIO13 (HW-PWM)	SG90 micro servo	SG90's plastic gear train is the common failure point under repeated full-lock steering — every scripted turn and wall-emergency escape drives it to the end of travel. MG90S's metal gears take that abuse in the same footprint and mounting, for a small weight and cost increase.
Battery	3 x 18650 Li-ion, series (~11.1 V)	2S/3S LiPo pack	18650 cells swap individually without a balance charger, and 11.1 V nominal clears the buck converter's dropout with margin without oversizing the pack.
Differential	Custom 3D-printed bevel differential	Solid axle (our first build) / off-the-shelf RC diff	A solid axle scrubbed the outer wheel through every corner and capped steering near 8°. An off-the-shelf diff didn't match the 25:20 gear pitch already tooled for the N20, so we designed our own.

4. Power and Sense Management

4.3 Wiring Diagram



4. Power and Sense Management

4.4 Sensing — Vision Pipeline

- Capture: the camera is read at 640x480 in RGB888, then the frame is rotated 180° in software, since the camera is physically mounted upside-down on its mast.
- Crop and resize: only the lower portion of the frame — the mat — is kept and resized down to 320x160 before any colour work happens, which both discards background clutter above the walls and keeps every frame cheap to process.
- Median blur, then HSV: a light blur is applied first because the camera runs a short exposure and elevated gain to freeze motion, which makes the raw image noisy enough to break both the colour masks and the wall scan if left untreated. The frame is then converted to HSV, which is far more stable than RGB under the mat's own lighting variation.
- Wall detection is a two-tier test, not one brightness threshold: a pixel is a wall if it is very dark regardless of colour, OR dark and low-saturation. A single brightness cut also lit up the blue and orange corner lines and the mat's printed markings, since they are dark too — and unevenly between the two frame halves, which injected a steering bias that looked like a control problem for a long time before we traced it to the measurement itself.
- Colour lines are searched only in the bottom of the region of interest, because a line painted on the mat physically cannot appear higher in the frame than the mat itself — searching the whole frame let bluish background pixels near the top trigger false line readings.

4. Power and Sense Management

4.4 Sensing — Camera Position

CAMERA POSITION IS CRUCIAL FOR THE ROBOT'S PERFORMANCE.

- Height 12.5 cm above the mat. The walls are 10 cm tall, so the lens must sit above the wall line to look down onto the mat and see the wall's base — the white-mat/black-wall edge, the highest-contrast feature in the frame.
- Tilt roughly 15° downward, enough to keep the mat and both wall bases in the lower frame while still seeing far enough ahead to catch a corner line or a traffic sign before reaching it.
- Centred laterally and facing dead ahead, zero yaw. Wall-following compares the left half of the frame against the right half, so any sideways offset or twist becomes a permanent steering bias that no amount of threshold tuning removes.
- Mounted on a rigid mast, not a flexible arm. A vibrating camera blurs the wall edge and injects noise straight into the steering loop, so mast stiffness is a control requirement, not a cosmetic one.
- The connection itself uses the Pi's dedicated CSI ribbon port, not a GPIO pin, so it needs no separate power and does not appear on the wiring diagram.

5. The Robot's Evolution

5.1 Past

Two decisions from earlier in the build turned out to be wrong in ways that only showed up once we started driving, not while designing.

The first camera was a Raspberry Pi Camera Module 3 — a larger sensor with autofocus. Focused on the mat it saw the near track sharply, but distant traffic signs blurred, and blur desaturates colour badly enough that the HSV masks lost far-away signs entirely; the car only 'saw' an obstacle once it was already close. A focus sweep confirmed we genuinely could not have both near and far sharp on that sensor at once. We replaced it with the OV5647, a smaller, fixed-focus sensor with a much larger depth of field — losing autofocus cost nothing, since a ground robot never needs to refocus.

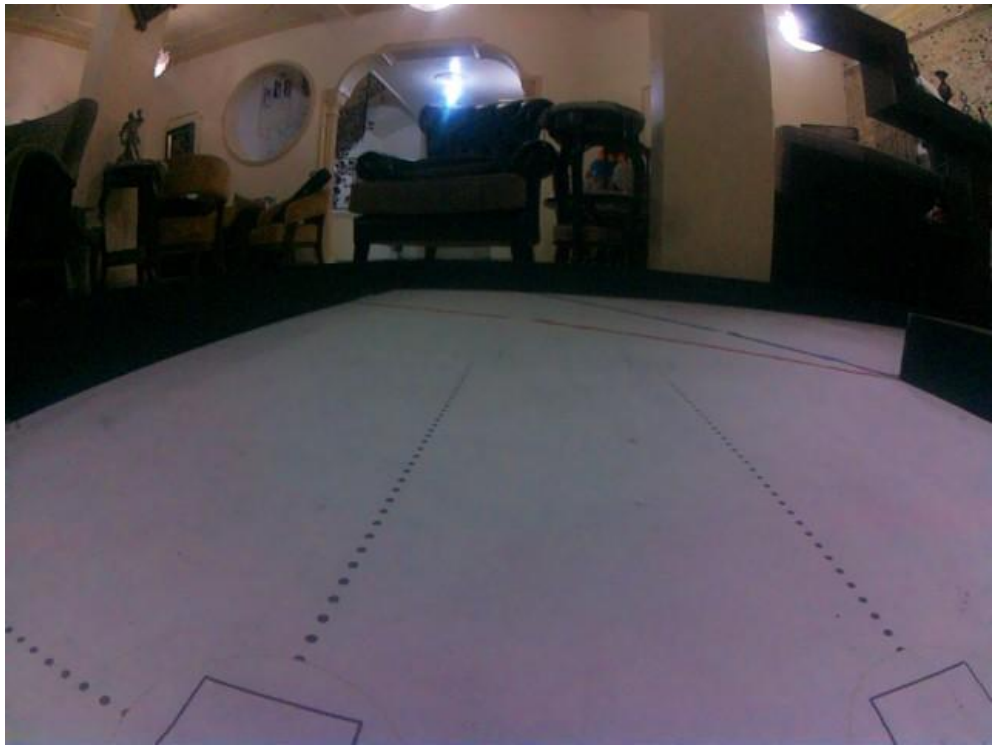
The second was the drivetrain. Our first rear axle was solid, so both driven wheels were forced to the same speed. In a corner the outer wheel has to cover more distance than the inner one, and with no way to do that it scrubbed, losing traction and sometimes stalling the car mid-turn. Our first fix was purely in software — cutting the steering limit down step by step until the scrubbing stopped, which landed near 8°. That kept the wheels turning but left the car unable to corner tightly enough to follow the track. It was a workaround for a mechanical problem, not a fix for it.

5. The Robot's Evolution

5.1a Camera Decision — Proof, Not Just a Claim

Same track, same lighting, same lens height — only the camera module changed. Both frames are straight off the Pi, unedited.

BEFORE — Camera Module 3 (autofocus), focused on the near mat



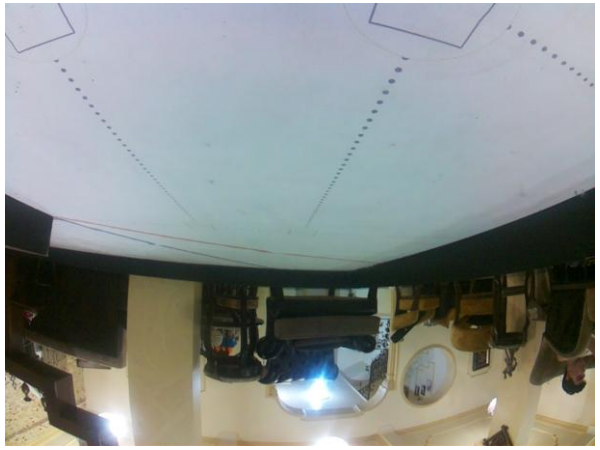
AFTER — OV5647 (fixed focus), same distance and framing

The Module 3's shallow depth of field could hold either the near mat or a distant sign in focus, never both — a focus sweep across f/2.0-equivalent apertures confirmed it wasn't a tuning problem. Blur desaturates colour enough that the HSV sign masks lost far signs entirely. The OV5647's smaller sensor and fixed focus trade autofocus (never needed by a ground robot) for a depth of field wide enough to keep the whole corridor sharp at once.

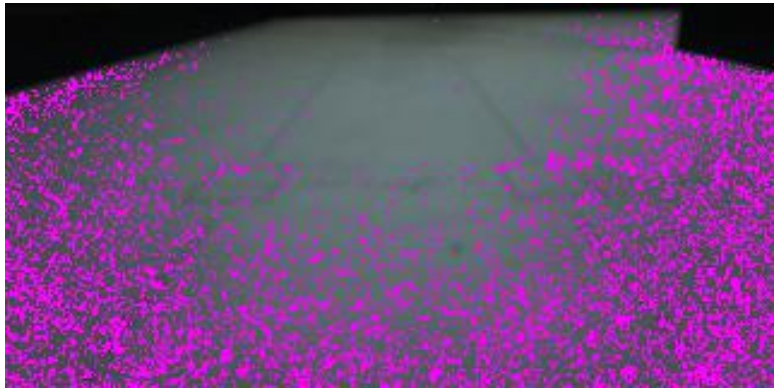
5. The Robot's Evolution

5.1b Vision Testing — Flip & Colour Tuning

Raw sensor — mounted upside-down

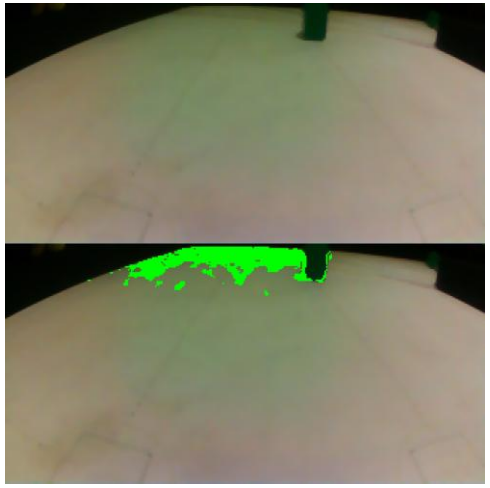


After the 180° software flip



HSV Value-channel scan, tuning

The camera mounts physically upside-down on its mast, so robot.py rotates every frame 180° before anything else touches it — real capture on the left, corrected on the right.



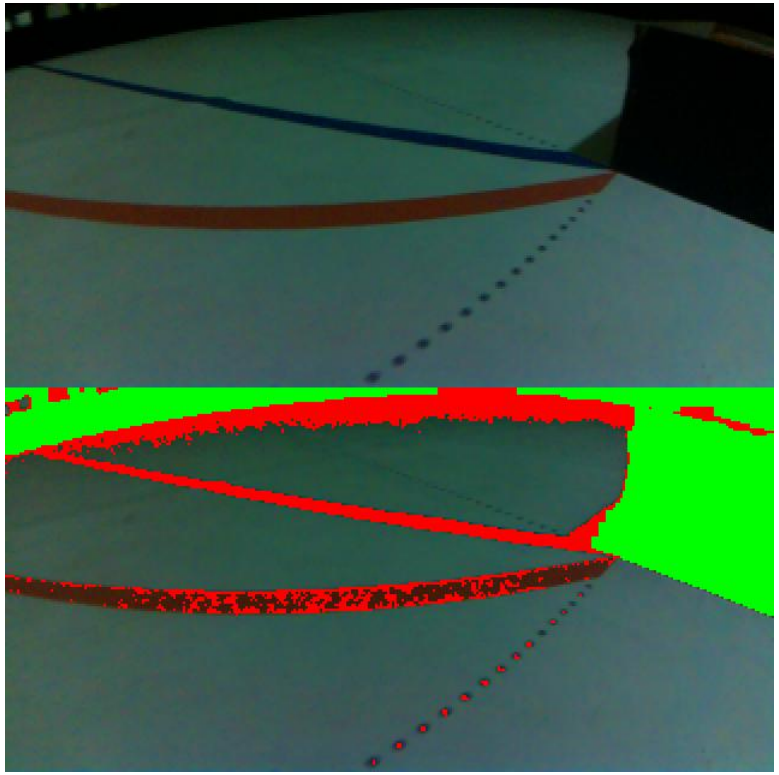
colors.json green-sign mask, live tuning

Colour thresholds are not guessed once and left alone — colour_tuner.py runs against the live camera so every HSV band (blue line, orange line, red sign, green sign, magenta gate) is set against the real mat under the real, locked exposure. Each iteration is checked against a visible mask overlay like the one on the left, so a threshold that looks right in isolation but bleeds onto the mat itself is caught before it reaches the car.

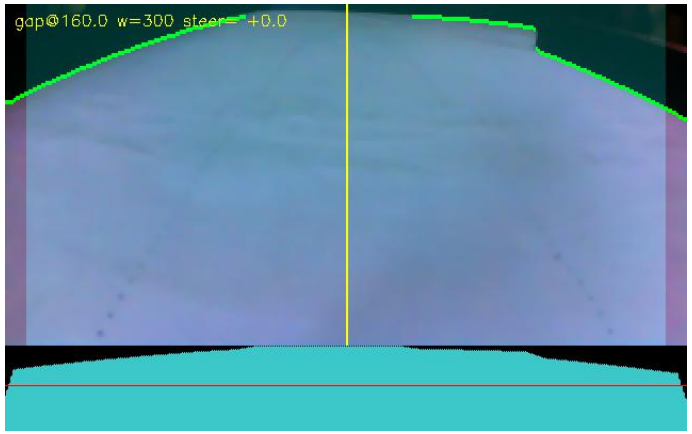
This is the same loop that caught the sign-detection bug in the Present section: the mat/wall boundary fringe read as green at S 96-111, merging with the real sign (S 135+) into one mis-shaped blob — visible only once we started saving the mask, not from the numbers alone.

5. The Robot's Evolution

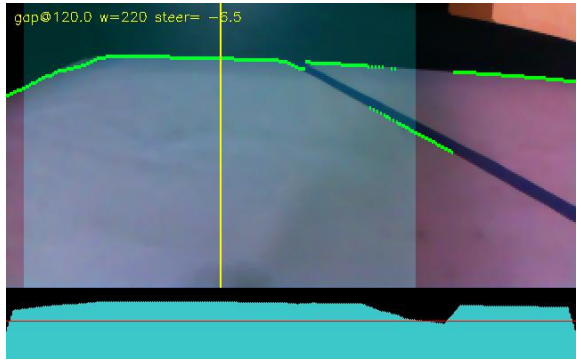
5.1c Vision Testing — Wall Masking & Distance Calibration



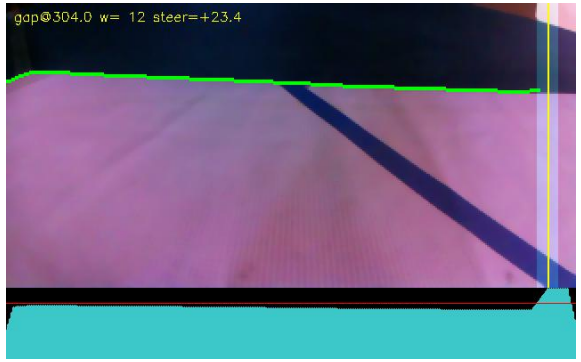
shadow_check.py — red = wall, green = free mat



freespace_test.py — gap, steer, boundary



Calibration at a taped 40 cm



Calibration at a taped 25 cm

Every distance constant in config.py traces back to frames like these: the car is placed at a taped, ruler-measured distance from the wall and freespace_test.py logs the wall-density reading at that exact spot — 40 cm measured 0.1032, 25 cm measured 0.1783 — so OUTER_TARGET is a real calibration curve, not a guessed constant. The left-hand overlay is the same diagnostic used to catch the wall-detector counting the blue/orange corner lines and the mat's own printed marks as wall (§9 of the engineering journal) — red pixels are what the code currently calls "wall".

5. The Robot's Evolution

5.2 Present

The current car fixes both of those at the source rather than around them.

A 3D-printed differential now sits on the rear axle, driven from the N20 motor through a 25:20 gear reduction. With the wheels free to turn at different speeds through a corner, the scrub is gone, and the steering limit came back up from 8° toward the linkage's mechanical ceiling.

The bigger present-day story, though, was perception rather than mechanics. Even after the camera swap, the car kept crashing near corners in both driving directions, in a way that no amount of gain or setpoint tuning changed. We eventually stripped the controller down to nothing and started saving an annotated frame every time the steering swung hard, colouring in every pixel the code was calling 'wall'. The answer was immediate: the wall detector was any dark pixel, and the blue line, the orange line, and the mat's own printed markings are all dark. They were being counted as wall — unevenly between the left and right halves of the frame — which is exactly why the fault only showed up near corners and resisted every attempt to tune it away. The controller had never been the problem; its input was lying to it.

Once the wall test was corrected to require genuine darkness or darkness plus low saturation, every distance constant in the project was re-measured from real centimetres, and the Open Challenge now completes in both directions with stable control.

5. The Robot's Evolution

5.3 Future

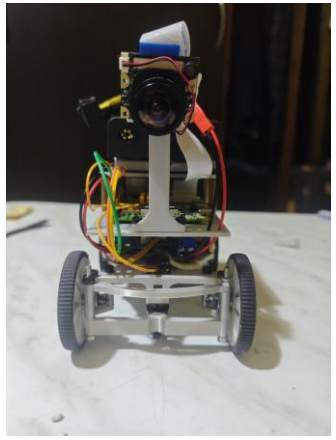
- Wire the parking manoeuvre into the Obstacle Challenge's main loop. The magenta gate is detected and currently treated purely as a wall to steer around; the reserved constants for the approach and stop condition already exist in config.py, but the behaviour itself is not yet connected.

- Close out a shadow-misreading issue seen at two of the four corners in field testing, where shadow on the mat is occasionally read as part of the wall. The existing two-tier wall test already rejects most shadow; a dedicated diagnostic tool (shadow_check.py) was built to separate a genuine wall from shadow in the field, and tightening the test against this case is active work.

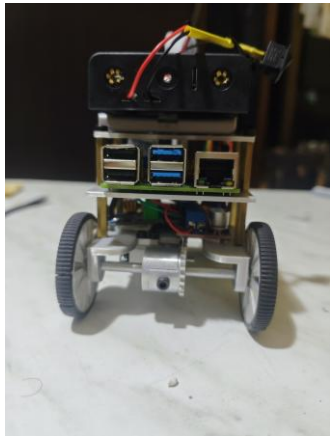
- Move wall-following from a pure proportional-derivative law on wall distance to a heading-plus-offset controller, so the car corrects its angle to the wall rather than only its distance from it — this should make the straights noticeably straighter.

- Track a sign's position across several frames instead of frame by frame, to smooth the approach further than the current short-memory hold already does.

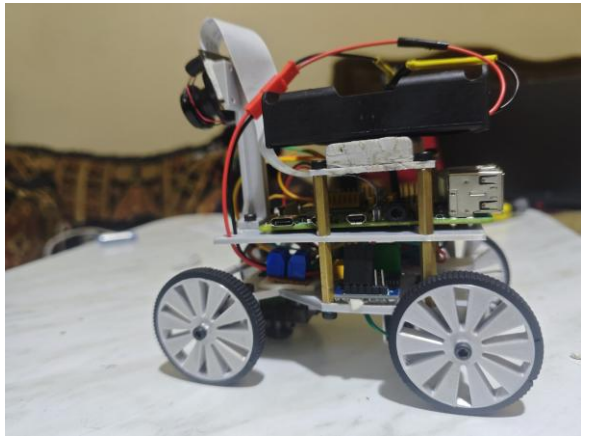
6. The Vehicle's Photos



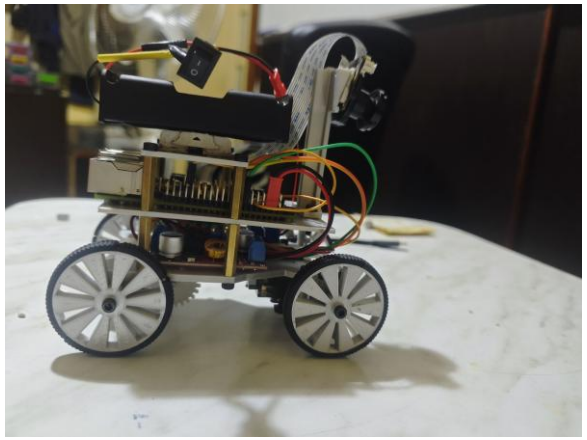
Front



Back

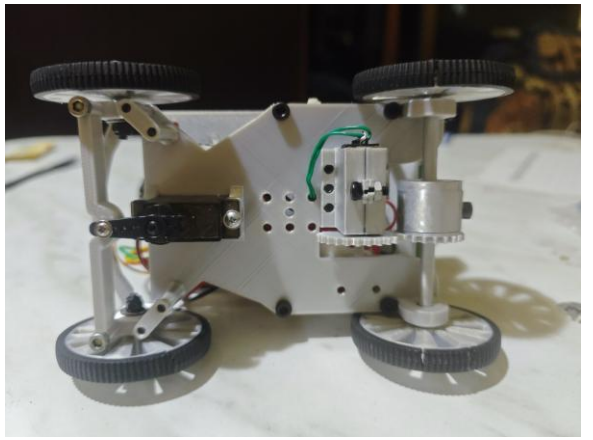


Left



Right

Top



Bottom

7. Team

Ahmad Kalthom

Coach

Computer and Automation Engineering student, Damascus University.

Jolian Wassof

Team member

Al-Awael School. AI, robotics and drones; WRO, Pacific and Syrian AI Olympiads.

Omar Shammout

Team member

IT Engineering student, Damascus University. Robotics, AI, electronics.

Louay Rashwan

Team member

Computer and Automation Engineering student, Damascus University.

8. Links and Credits

- GitHub repository (full source, CAD, documentation):
  - https://github.com/jolianjij/Red-Castle-WRO-Future-Engineers-2026
- Open Challenge driving video:
  - https://youtu.be/GmrX7NhcAxE
- Obstacle Challenge driving video:
  - https://youtu.be/ujbImlpQH6g

Thanks

As a team, we thank HMK AI and Robotics Club for hosting and supporting us throughout this build, and our coach, Ahmad Kalthom, for the guidance that kept the project moving through every dead end along the way.